



**SRI MANAKULA VINAYAGAR**  
**ENGINEERING COLLEGE**

**(AN AUTONOMOUS INSTITUTION)**

(APPROVED BY AICTE, NEW DELHI AND AFFILIATED TO PONDICHERRY UNIVERSITY)  
(ACCREDITED BY NBA-AICTE, NEW DELHI, ACCREDITED BY NAAC WITH 'A' GRADE)

MADAGADIPET, PUDUCHERRY - 605 107



## **DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND DATA SCIENCE**

### **ASSIGNMENT – 2**

---

**Course Title : Design and Analysis of  
Algorithms**

**Course Code : U23CSBC01**

**NAME : Vasanth S**

**REG NO : 24UAI172**

---

**Answer all questions.**

What is a multistage graph in Dynamic Programming? (CO2, K3)

Define the 0/1 Knapsack problem. (CO2, K3)

What is the general approach used in backtracking algorithms? (CO3, K3)

State the N-Queen problem in brief. (CO3, K3)

What is the Traveling Salesperson Problem (TSP)? (CO2, K3)

✂ Part B – Descriptive/Practical Questions (2 x 5 = 10 Marks)

Answer any two questions.

1. Using Dynamic Programming, solve the 0/1 Knapsack problem for the following items:

Weights: [2, 3, 4, 5]

Values: [3, 4, 5, 6]

Knapsack Capacity: 5

Show the steps and explain your solution. (CO2, K3)

Practical:

Implement the N-Queen problem for N=4 using Backtracking and show all possible solutions. Explain how backtracking helps prune the search space. (CO3, K3)

→ Submit: Screenshot of code and output.

Solve the All Pairs Shortest Path problem for the following weighted graph using Dynamic Programming (Floyd - Warshall Algorithm). Provide the distance matrix after each iteration.

Graph:

Vertices: {A, B, C, D}

Edges with weights: A→B=3, A→C=8, B→C=2, B→D=5, C→D=1 | CO2 | K3 ...MAKE THESE MORE

DETAILED ANSWER

Here are detailed, exam-ready answers with clear steps, definitions, and worked examples for all requested items, expanded while remaining concise and easy to follow.<sup>[1][2][3][4][5][6][7][8]</sup>

## **Part A — Short Answers**

### **Multistage graph in DP**

A multistage graph is a directed, weighted graph whose vertices are partitioned into ordered stages, and edges go only from one stage to the immediately next stage, modeling sequential decision processes over phases. Dynamic programming computes optimal costs or paths by propagating solutions across stages (often backward from the last stage), since each stage's optimal choice depends only on optimal choices in adjacent stages along allowable edges.<sup>[2][9][5][1]</sup>

### **0/1 Knapsack definition**

The 0/1 Knapsack problem gives items with weights and values and a knapsack capacity  $W$ , asking to select a subset that maximizes total value without exceeding  $W$ , where each item can be either taken whole or not taken at all (no fractions). The classic DP recurrence considers for item  $i$  and capacity  $w$  the better of excluding the item or including it (if it fits), building a table  $dp[i][w]$  for optimal substructure and overlapping subproblems.<sup>[10][11][12]</sup>

### **General backtracking approach**

Backtracking incrementally builds partial candidates in a depth-first manner and prunes a branch as soon as it cannot possibly lead to a valid or optimal solution, then reverses decisions to explore alternatives. Conceptually, it traverses a search tree of partial solutions, using constraint checks and heuristics to cut off failing branches early to reduce the search space.<sup>[3][6][13]</sup>

### **N-Queen problem (brief)**

The N-Queen problem asks to place  $N$  queens on an  $N \times N$  chessboard so that no two queens share a row, column, or diagonal, typically solved by backtracking with one queen per row and safety checks per placement. Constraint propagation eliminates attacked positions after each placement, and backtracking moves a queen when a row cannot be completed safely, enabling efficient enumeration of solutions.<sup>[14][7][8]</sup>

### **Traveling Salesperson Problem (TSP)**

TSP seeks the shortest Hamiltonian tour that visits each city exactly once and returns to the start, given intercity distances, and it is NP-hard with reductions from Hamiltonian cycle showing its complexity. Exact methods (e.g., DP Held–Karp) solve small instances, while heuristics/approximations are used for larger scales due to combinatorial explosion.<sup>[4]</sup>

## Part B — Descriptive/Practical

### 1) 0/1 Knapsack via Dynamic Programming

- Items: weights , values , capacity  $W = 5$ , with 0/1 choice per item only.<sup>[9][15][16][11][2][10]</sup>
- Define  $dp[i][w]$  = max value using first  $i$  items with capacity  $w$ ; initialize  $dp[w] = 0$  and  $dp[i] = 0$  for all  $i, w$ .<sup>[11][10]</sup>
- Recurrence for item  $i$  (1-indexed) with weight  $w_i$  and value  $v_i$ :  
If  $w_i \leq w$ ,  $dp[i][w] = \max(dp[i-1][w], dp[i-1][w-w_i] + v_i)$ ; else  $dp[i][w] = dp[i-1][w]$ .<sup>[10][11]</sup>

Progression table (rows:  $i=0..4$ , cols:  $w=0..5$ ) shows computed optima at each subproblem :<sup>[11][10]</sup>

- Row  $i=1$  ( $w_1=2, v_1=3$ ):  $dp$  becomes for  $w=0..5$  as item1 fits when  $w \geq 2$  and improves value 3.<sup>[9][10][11]</sup>
- Row  $i=2$  ( $w_2=3, v_2=4$ ): at  $w=3$ , choose  $\max(3, 0+4)=4$ ; at  $w=5$ , choose  $\max(3, dp+4=3+4=7) = 7$ , so  $dp$  is for A's row.<sup>[15][17][11][9][3][10][11]</sup>
- Row  $i=3$  ( $w_3=4, v_3=5$ ): at  $w=4$ ,  $\max(4, 0+5)=5$ ; at  $w=5$ ,  $\max(7, dp+5=0+5=5)=7$ , so row becomes.<sup>[16][17][11][9][15][3][10][11]</sup>
- Row  $i=4$  ( $w_4=5, v_4=6$ ): at  $w=5$ ,  $\max(7, dp+6=6)=7$ ; final row but the best at capacity 5 is carried from above as 7.<sup>[2][9][15][16][10][11]</sup>

Final  $dp$  table (0..5 capacity):

- $i=0$ : 0 0 0 0 0 0
- $i=1$ : 0 0 3 3 3 3
- $i=2$ : 0 0 3 4 4 7
- $i=3$ : 0 0 3 4 5 7
- $i=4$ : 0 0 3 4 5 7 (effective best retained).<sup>[10][11]</sup>

Answer: optimal value is 7 using items of weight 2 and 3 (values 3 and 4), exactly filling capacity 5, with reconstruction by tracing choices where include-case was selected at  $(i, w)$ .<sup>[11][10]</sup>

## Practical: N-Queen (N=4) with Backtracking

- Strategy: place one queen per row; for row  $r$ , try columns  $c$  that are safe given earlier rows, checking column and diagonals ( $c$ ,  $r-c$ ,  $r+c$ ), and recurse; if no column works, backtrack to previous row and move its queen to the next safe column.<sup>[7][8][14]</sup>
- Pruning: after each placement, eliminate attacked squares in future rows; this reduces branching drastically and prevents exploring impossible partial boards, a standard backtracking pruning efficiency gain.<sup>[6][8][3]</sup>

All solutions for N=4 (two distinct configurations):

1)

. Q . .

. . . Q

Q . . .

. . Q .

(Positions: (1,2), (2,4), (3,1), (4,3)).<sup>[8][14][7]</sup>

2)

. . Q .

Q . . .

. . . Q

. Q . .

(Positions: (1,3), (2,1), (3,4), (4,2)).<sup>[14][7][8]</sup>

Illustrative Python backtracking (for code screenshot):

- Maintain sets for used columns and diagonals; recurse row-wise constructing a board array; append full boards to a solutions list when  $\text{row} == N$ ; backtrack by removing last placement to try next column.<sup>[8][14]</sup>
- Output both boards in human-readable grid form matching the two configurations above for submission.<sup>[14][8]</sup>

Backtracking benefit: without pruning, naive enumeration explores  $4^4$  assignments; backtracking with safety checks prunes most branches early, scaling to larger  $N$  efficiently compared to brute force.<sup>[3][6][8]</sup>

## Floyd–Warshall: All-Pairs Shortest Paths

Graph:  $V=\{A,B,C,D\}$ ; edges  $A\rightarrow B=3$ ,  $A\rightarrow C=8$ ,  $B\rightarrow C=2$ ,  $B\rightarrow D=5$ ,  $C\rightarrow D=1$ ; initialize dist with 0 on diagonals, given weights, and  $\infty$  where no direct edge exists.<sup>[1]</sup>

Initial  $D(0)$  with vertex order  $[A,B,C,D]$  as intermediates:

- A:  $[0, 3, 8, \infty]$
- B:  $[\infty, 0, 2, 5]$
- C:  $[\infty, \infty, 0, 1]$
- D:  $[\infty, \infty, \infty, 0]$ .<sup>[1]</sup>

Iteration  $k=A$ : using  $A$  as intermediate yields no improvements because  $A$  does not shorten any paths that start elsewhere or improve existing ones beyond direct arcs already minimal here.<sup>[1]</sup>

Matrix after  $k=A$  remains:

A:  $[0,3,8,\infty]$ ; B:  $[\infty,0,2,5]$ ; C:  $[\infty,\infty,0,1]$ ; D:  $[\infty,\infty,\infty,0]$ .<sup>[1]</sup>

Iteration  $k=B$ : check  $i\rightarrow B\rightarrow j$  paths;  $A\rightarrow C$  improves via  $B$ :  $3+2=5 < 8$ ;  $A\rightarrow D$  improves via  $B$ :  $3+5=8 < \infty$ ; others unchanged, producing:

A: ; B:  $[\infty,0,2,5]$ ; C:  $[\infty,\infty,0,1]$ ; D:  $[\infty,\infty,\infty,0]$ .<sup>[9][16][14][1]</sup>

Iteration  $k=C$ : use  $C$  as intermediate;  $A\rightarrow D$  improves via  $C$ :  $5+1=6 < 8$ ;  $B\rightarrow D$  improves via  $C$ :  $2+1=3 < 5$ ; others unchanged:

A: ; B:  $[\infty,0,2,3]$ ; C:  $[\infty,\infty,0,1]$ ; D:  $[\infty,\infty,\infty,0]$ .<sup>[16][9][10][1]</sup>

Iteration  $k=D$ :  $D$  has no outgoing paths that reduce any current distances, so the matrix remains final:

A: ; B:  $[\infty,0,2,3]$ ; C:  $[\infty,\infty,0,1]$ ; D:  $[\infty,\infty,\infty,0]$ .<sup>[9][16][10][1]</sup>

Rationale: Floyd–Warshall iteratively refines  $d[i][j]$  with  $d[i][k]+d[k][j]$  across all  $k$ , guaranteeing all-pairs shortest paths in  $O(n^3)$  time on directed weighted graphs with non-negative and even negative edges (if no negative cycles).<sup>[1]</sup>

## Quick reference table

| Topic            | Key points                                                                                                                                  |
|------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| Multistage graph | Directed stages; edges only to next stage; DP computes optimal stage-to-stage paths via forward/backward propagation <sup>[1][2]</sup>      |
| 0/1 Knapsack     | Maximize value under capacity with 0/1 item choices; DP recurrence includes/excludes items with $dp[i][w]$ <sup>[11][10]</sup>              |
| Backtracking     | DFS over partial candidates; prune failing branches; reverse choices to explore alternatives efficiently <sup>[3][6]</sup>                  |
| N-Queen          | Place N queens with no attacks; solved by backtracking with constraint checks and propagation; N=4 has 2 solutions <sup>[8][14]</sup>       |
| TSP              | Shortest tour visiting each city once; NP-hard via reduction from Hamiltonian cycle; exact feasible only for small instances <sup>[4]</sup> |

These expanded explanations, step-by-step tables, and algorithmic rationales match academic expectations for detailed answers while remaining structured and direct for quick study and submission.<sup>[2][4][3][8][10][14][11][1]</sup>

\*  
\*\*